STM32 EMBEDDED FIRMWARE | REGISTER-LEVEL DRIVERS

Reusable Bare-Metal SPI Functions

A practical function reference for direct-register STM32 projects: configuration, chip select, full-duplex transfers, buffers, status flags and error handling.

SPI Protocol	Bare metal No HAL calls	Reusable API Driver + application

What you will learn

Use this guide as a reusable function reference when building embedded projects in STM32CubeIDE. Keep the low-level bus driver separate from application-specific work such as reading a sensor, controlling a motor or storing data.

Common functions

SPI_Init() - Configure GPIO pins, clock, master/slave, data size, bit order, CPOL/CPHA and prescaler.

SPI_SelectDevice() / SPI_DeselectDevice() - Control the target device chip-select (CS/NSS) line.

SPI_WaitFlag() - Wait for TXE, RXNE or BSY with a timeout.

SPI_TransferByte() - Transmit a byte and receive the simultaneously shifted byte.

SPI_SendByte() / SPI_ReadByte() - Convenience wrappers around full-duplex transfer.

SPI_WriteBuffer() - Transmit a byte array while selected.

SPI_ReadBuffer() - Generate clock pulses and collect received bytes.

SPI_TransferBuffer() - Transmit and receive a buffer at the same time.

SPI_ClearErrors() / SPI_GetStatus() - Handle/report overrun and mode/configuration errors.

SPI_TransactionBegin() / SPI_TransactionEnd() - Group a device command and payload under one CS assertion.

Important: SPI registers, alternate-function mappings and status-flag clearing vary across STM32 families. The API concepts are reusable, but the register implementation must match your exact MCU and target device datasheet.

1. Understand SPI

SPI is a synchronous serial bus commonly using SCK (clock), MOSI (controller-to-peripheral data), MISO (peripheral-to-controller data) and CS/NSS (device select). Unlike I2C, SPI has no standard address byte or ACK bit; the selected peripheral and command format are defined by the board and device.

Signal	Typical direction	Purpose
SCK	Controller -> peripheral	Clock generated by controller in master mode.
MOSI	Controller -> peripheral	Data sent to the selected device.
MISO	Peripheral -> controller	Data returned from the selected device.
CS/NSS	Controller -> peripheral	Selects the device, often active low.

2. Function list - what to reuse

Function	Responsibility	Reuse guidance
SPI_Init()	Enable peripheral clock; configure GPIO, master/slave, baud prescaler, CPOL/CPHA and frame size.	API reusable; registers/pins/clock are MCU-specific.
SPI_SelectDevice()	Drive the selected device CS low (or use hardware NSS where appropriate).	Board/device-specific CS pin mapping.
SPI_DeselectDevice()	Release CS after the complete command/data transaction.	Common helper; respect the target datasheet timing.
SPI_WaitFlag(flag, timeout)	Wait for TXE/RXNE/BSY or another status condition with timeout.	Pattern reusable; flags differ across SPI IP versions.
SPI_TransferByte(tx, *rx)	Transmit and receive one frame simultaneously.	Core SPI helper; full-duplex RX must be drained.
SPI_SendByte(byte)	Send a byte and handle/discard the simultaneous received byte.	Convenience wrapper.
SPI_ReadByte(dummy, *rx)	Clock data in by transmitting a dummy byte/frame.	Dummy value is device/protocol-specific.
SPI_WriteBuffer(data, len)	Transmit multiple frames.	Reusable buffer API.
SPI_ReadBuffer(data, len)	Transmit dummy frames and store received frames.	Reusable buffer API.
SPI_TransferBuffer(tx, rx, len)	Exchange arrays in full duplex.	Reusable when the target protocol needs both directions.
SPI_GetStatus() / SPI_ClearErrors()	Report and handle overrun, mode fault and other errors.	Register semantics are MCU-specific.
SPI_TransactionBegin/End()	Keep CS active for command, address and payload phases.	Adapt to each target device CS timing rules.

3. Typical SPI transaction

The controller normally asserts CS, transmits the command/address, clocks the required data, waits until the transfer has completed and then releases CS. The exact command bytes and dummy cycles are target-specific.

Step	Action	Why
1	Configure SPI mode and clock	Match the peripheral datasheet: CPOL, CPHA, frame size and maximum SCK.
2	Assert CS	Select one peripheral before sending a command.
3	Send command/address frames	Tell the device which operation or register is requested.
4	Transfer payload / dummy frames	Every transmitted frame also clocks a received frame in full duplex.
5	Wait until not busy	Ensure the final frame has completed on the wire.
6	Deassert CS	End the device-specific transaction.

Example reusable API (interface only)

```
typedef enum { SPI_OK, SPI_TIMEOUT, SPI_OVERRUN, SPI_MODE_FAULT } SPI_Status;

void SPI_Init(void);
void SPI_SelectDevice(void);
void SPI_DeselectDevice(void);
SPI_Status SPI_TransferByte(uint8_t tx, uint8_t *rx);
SPI_Status SPI_WriteBuffer(const uint8_t *tx, size_t len);
SPI_Status SPI_ReadBuffer(uint8_t *rx, size_t len, uint8_t dummy);
SPI_Status SPI_TransferBuffer(const uint8_t *tx, uint8_t *rx, size_t len);
```

Critical full-duplex rule

SPI shifts data in both directions on every clock. Even a transmit-only operation receives a frame; read/drain the receive data register as required by the MCU reference manual or an overrun condition can occur. During a read, transmit a dummy frame to generate clock pulses.

SPI mode is not optional

CPOL determines the idle clock level; CPHA determines which clock edge samples data. SPI modes 0-3 are combinations of CPOL and CPHA. Match the exact device datasheet. Also verify bit order, maximum SCK frequency, CS setup/hold time, and required dummy clocks.

4. Portable design vs MCU-specific code

Usually reusable	Must be checked for each STM32
Public APIs; buffer-transfer concepts; CS transaction helpers; timeout policy; device-independent status codes.	RCC clock enables; GPIO AF mapping; SPI instance; register bits; baud prescaler; CPOL/CPHA encoding; frame size; TXE/RXNE/BSY semantics; error-flag clearing; DMA/interrupt details.

5. Common SPI problems to troubleshoot

Symptom	Things to check
All bytes shifted / wrong values	Wrong CPOL/CPHA, bit order, word size or sampling edge.
No response from device	CS wiring/polarity, wrong command, wrong SPI mode, absent power or missing common ground.
Overrun error	Receive data not read in time; drain RX on every transmitted frame as the MCU requires.
Intermittent operation	SCK too fast, poor wiring, wrong CS timing, signal integrity, or bus contention.
Code hangs	Busy/flag polling has no timeout, wrong peripheral clock assumption, or an error flag blocks progress.

6. Recommended project structure

```
Core/Inc/spi_driver.h      // API, status codes, documented frame format
Core/Src/spi_driver.c      // registers, flags, transfers, timeout, error handling
App/Src/flash.c            // flash-specific commands and memory operations
App/Src/display.c          // display-specific commands and drawing
Core/Src/main.c            // initialize driver and run application tasks
```

Official references

STMicroelectronics RM0008 - STM32F1 reference manual (SPI chapter):
https://www.st.com/resource/en/reference_manual/cd00171190-stm32f101-103-105-107-stm32f100-series-armbased-32bit-mcus-stmicroelectronics.pdf

Check the datasheet for the peripheral device as well as the MCU reference manual. SPI protocol commands and timing are usually device-specific.